Chapter 3

Software Quality Requirements

After completing this chapter, you will be able to:

- describe the history of the concepts conveyed by software quality models;
- understand the different characteristics and sub-characteristics of software quality as outlined in the ISO 25010 international standard;
- use the concepts to specify the software quality requirements of a software product;
- explain the positive and negative interaction between the quality characteristics of software as described in the ISO 25010 international standard;
- understand the concept of software traceability.

3.1 INTRODUCTION

All software is an element of a system, whether it be a computer system in which the software may be used on a personal computer, or in an electronic consumer product like a digital camera. The needs or requirements of these systems are typically documented either in a request for quote (RFQ) or request for proposal (RFP) document, a statement of work (SOW), a software requirements specification (SRS) document or in a system requirements document (SRD). Using these documents, the software developer must extract the information needed to define specifications for both the functional requirements and performance or non-functional requirements required by the client. The term "non-functional," as applied to requirements, is deprecated and is not used in the Institute of Electrical and Electronics Engineers (IEEE) 730 standard [IEE 14].

Software Quality Assurance, First Edition. Claude Y. Laporte and Alain April.
© 2018 the IEEE Computer Society, Inc. Published 2018 by John Wiley & Sons, Inc.

Functional Requirement

A requirement that specifies a function that a system or system component must be able to perform.

ISO 24765 [ISO 17a]

Non-Functional Requirement

A software requirement that describes not what the software will do but how the software will do it. Synonym: design constraint.

ISO 24765 [ISO 17a]

Performance Requirement

The measurable criterion that identifies a quality attribute of a function or how well a functional requirement must be accomplished (IEEE Std 1220™-2005). A performance requirement is always an attribute of a functional requirement.

IEEE 730 [IEE 14]

Software quality assurance (SQA) must be able to support the practical application of these definitions. To achieve this, many concepts proposed by software quality models must be mastered. This chapter is dedicated to presenting the models as well as the software engineering standards available for correctly defining performance or non-functional (i.e., quality) requirements of software. Using these SQA practices early in the software development life cycle will ensure that the client receives a quality software product that meets his needs and expectations.

Quality Model

A defined set of characteristics, and of relationships between them, which provides a framework for specifying quality requirements and evaluating quality.

ISO 25000 [ISO 14a]

The above definition of a quality model implies that the quality of software can be measured. In this chapter, we describe the research carried out over the years that culminated in a definition of a software quality model. Quality assurance, in certain industries and business models overseen by the software developer, requires a more formal management of software quality throughout the software life cycle. If it is not possible to evaluate the quality of the resulting software, how can the client be in a

position to accept it? Or at least, how can we prove that the quality requirements were met?

In order to support this condition, a software quality model is used so that the client can:

- define software quality characteristics that can be evaluated;
- contrast the different perspectives of the quality model that come into play (i.e., internal and external perspectives);
- carefully choose a limited number of quality characteristics that will serve as the non-functional requirements for the software (i.e., quality requirements);
- set a measure and its objectives for each of the quality requirements.

It is therefore necessary for the model to demonstrate its ability to support the quality requirements definition, and subsequently, their measurement and evaluation. We have seen in the previous chapter that quality is a complex concept. Quality is often measured by considering a specific perception of quality. Humankind has mastered measuring physical objects for centuries, but even today, many questions remain about measuring a software product and the ability to measure it in an objective manner. The role and importance of each quality characteristic are still difficult to clearly define, identify, and isolate. What's more, software quality is often a subjective concept that is perceived differently depending on the point of view of the client, user, or software engineer.

Evaluation

A systematic examination of the extent to which an entity is capable of fulfilling specified requirements.

ISO 12207 [ISO 17]

This chapter will provide the SQA practitioner with the knowledge to use the ISO 25010 software quality model concepts. In this way, he can initiate processes and support software engineers in their development, maintenance, and software acquisition projects. We will begin by presenting the history of the different models and standards designed to characterize software quality. This will be followed by a discussion of the concept of software criticality and its value. We will then introduce the concept of quality requirements and present a process for defining quality requirements. Lastly, the technique of software traceability, which assures that a requirement has truly been integrated into the software, will be outlined. In a later chapter, we will cover traceability in greater detail.

Chapter 4

Software Engineering Standards and Models

After completing this chapter, you will be able to:

- understand the evolution and the importance of software engineering standards for the SQA specialist;
- understand the standards for software quality management systems: ISO 9001, ISO/IEC 90003, ISO/IEC 20000, and the TickIT certification process [TIK 07];
- understand the software engineering standards: ISO/IEC/IEEE 12207 and IEEE 730 that govern the content of the SQA plan;
- include other improvement models, norms, standards, and quality processes: CMMI® maturity models of software processes and S^{3m}, ITIL, the CobiT IT governance approach, the ISO/IEC 27000 information security standard, and the ISO/IEC 29110 standards for very small organizations;
- understand that there are also repositories and standards specific to certain application domains: DO-178 and ED-12 for aeronautics, EN 50128 for railways and ISO 13485 for the field of medical devices that contain software;
- understand the importance of standards in terms of SQA.

4.1 INTRODUCTION

Other engineering domains such as mechanical, chemical, electrical, or physics engineering are based on the laws of nature as discovered by scientists. Figure 4.1 illustrates some of the many laws of nature.

Unfortunately, software engineering, unlike other engineering disciplines, is not based on the laws of nature. This explains, in part, some of the setbacks discussed in Chapter 1. The number of defective software, accidents and deaths, projects over

Software Quality Assurance, First Edition. Claude Y. Laporte and Alain April.
© 2018 the IEEE Computer Society, Inc. Published 2018 by John Wiley & Sons, Inc.

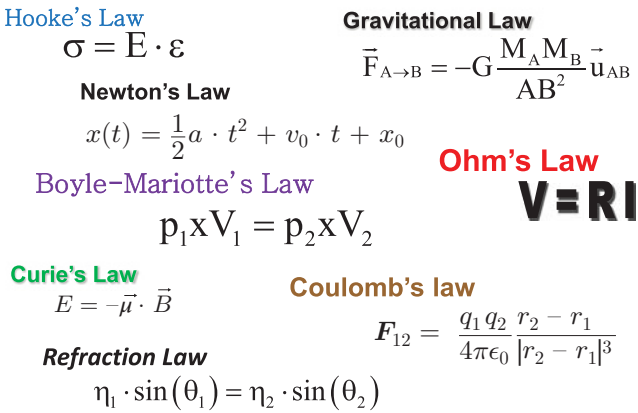

Figure 4.1 A few laws of nature used by some engineering disciplines.

budget and that are delivered past deadlines, and frustrated users are some examples of these setbacks.

The development of software is based only on the laws of logic and mathematics. Software engineering, like other disciplines, is based on the use of well-defined practices for ensuring the quality of its products. In software engineering, there are several standards, which are actually guides for management practices. A rigorous process is the framework for the way standards are developed and approved including, among others, international ISO standards and standards from professional organizations such as IEEE.

The four principles for the development of ISO standards are:

– ISO standards meet a market need.

– ISO standards are based on worldwide expertise.

– ISO standards are the result of a multi-stakeholder process.

– ISO standards are based on consensus.

The ISO standards are developed by consensus, as defined below. Consensus is required to produce a standard that will be accepted by the community of interest.

Consensus

General agreement characterized by the absence of sustained opposition to substantial issues by any important part of the concerned interests and by a process that involves

seeking to take into account the views of all parties concerned and to reconcile any conflicting arguments.

Note: Consensus need not imply unanimity.

ISO Directive, Part 1

Consensus means (adapted from Coallier (2003) [COA 03]):

- That all parties were able to express their views;
- The best effort has been made to take into account all opinions and solve all problems (i.e., all the submissions in a vote of the draft of a standard).

Standard

A set of mandatory requirements established by consensus and maintained by a recognized body to prescribe a disciplined and uniform approach, or to specify a product, with respect to mandatory conventions and practices.

ISO 24765 [ISO 17a]

The first condition to bear in mind is that standards, which differ from other written guidelines, are quasi-legal documents. Standards can be used either to prove or refute elements in a court of law. Standards often become legal requirements when they are adopted by governments and regulatory agencies. When this occurs, the content of a standard is important, since organizations then use it to develop products and services that can greatly impact human life, the environment, or business.

"Organizations formalize behavior to reduce its variability, and, ultimately, to predict it and control it."

Mintzberg (1992) [MIN 92]

The right-hand side of Figure 4.2 shows the development of standards as of the 1970s when the American Department of Defense (DoD) created the "DoD-STD-1679A" military standard [DOD 83]. At that time, a contract was given to a supplier

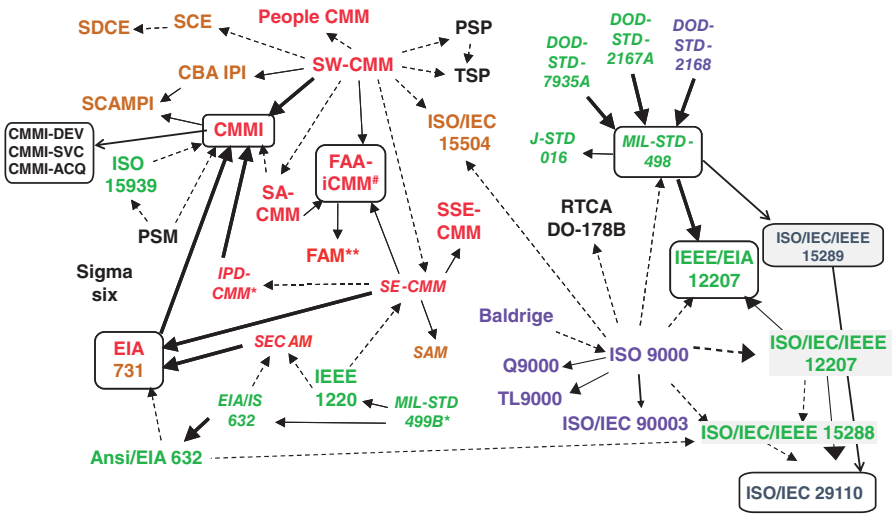

Figure 4.2 The development of standards and models.
Source: Adapted from Sheard (2001) [SHE 01] with the permission of Systems and Software Consortium, Inc.

to develop a piece of software, and it took months, if not a year before receiving the final product. Since the client did not see any development with this process, and, in the end, received boxes containing documents and magnetic tape, this approach was called the "*Big Bang*." The DoD then decided that the entire software development process had to become more transparent to allow documents produced throughout the development cycle to be evaluated. The military standard required that the supplier write and have a number of documents approved. These approvals allowed clients to review, comment, and approve documents instead of waiting until the end and receiving software that did not meet their needs. Review and approval activities for documents were related to certain project management activities: the approval of these documents led to the payment of a large sum of money as agreed to in the contract with the supplier. This enabled the client to remotely control the development of the requested software. Over the years, other organizations such as the IEEE, the International Organization for Standardization (ISO), and the European Space Agency (ESA) developed standards. During the late 1980s, the American DoD decided to use commercial standards instead of military standards, such as ISO/IEC/IEEE 12207 [ISO 17], to develop its software. Military software engineering standards were then removed.

On the left-hand side of the figure, we note the "*Capability Maturity Model*" (CMM®). It was developed at the request of the American DoD, by the Software Engineering Institute (SEI) in order to provide a road map of engineering practices to improve the performance of the development, maintenance and service provisioning processes. This model is described in a later section.

The Continuous Evolution of Standards

The standards presented in this book are constantly evolving. They are reviewed periodically and, if necessary, updated. Some standards are updated about every 5 years. Others are updated if major changes are needed. In an organization, it is possible that different versions of the same standard are used. For example, when an agreement or a contract is signed, we normally quote the latest version of the standard in the contract. In some organizations, such as those working in military defense and aerospace, a development or maintenance project can span over many years if not several decades. In such cases, it is possible that the customer prefers to continue the project with the version of the standard used at the beginning of the project, while for the same organization, a new project will use the latest version of the same standard. Developers and SQA must then carry out their responsibilities with the two versions of the same standard and use templates and checklists specific to each version.

Figure 4.3 illustrates the evolution of standards that are maintained and published under the responsibility of the appointed subcommittee for standardized processes, tools, and supporting technologies for software engineering and systems: ISO/IEC JTC1 SC7.

The ISO Name

Because the name "International Organization for Standardization" would have resulted in different abbreviations in different languages (e.g. "IOS" in English and "OIN" in French), the founders opted for a short name "ISO". This name is derived from the Greek "ISOS", meaning "equal". Whatever the country, whatever the language, the short form of the name of the organization is therefore always ISO.

ISO website (https://www.iso.org/about-us.html)

In the 1980s, there were only five standards; in 2016, more than 160 standards make up the portfolio of sub-committee 7 (SC7). This rapid increase is due, among others, to the fact that more software engineering practices have matured and acquired a broad consensus since the late 1980s.

More than 39 countries actively participate in the development of SC7 standards and 20 countries participate as observers. For countries that actively participate and have the right to vote, the meaning of the Greek word "ISOS" implies that the vote of any ISO member country is equal to the vote of any other country, no matter its size, economic, or political influence.

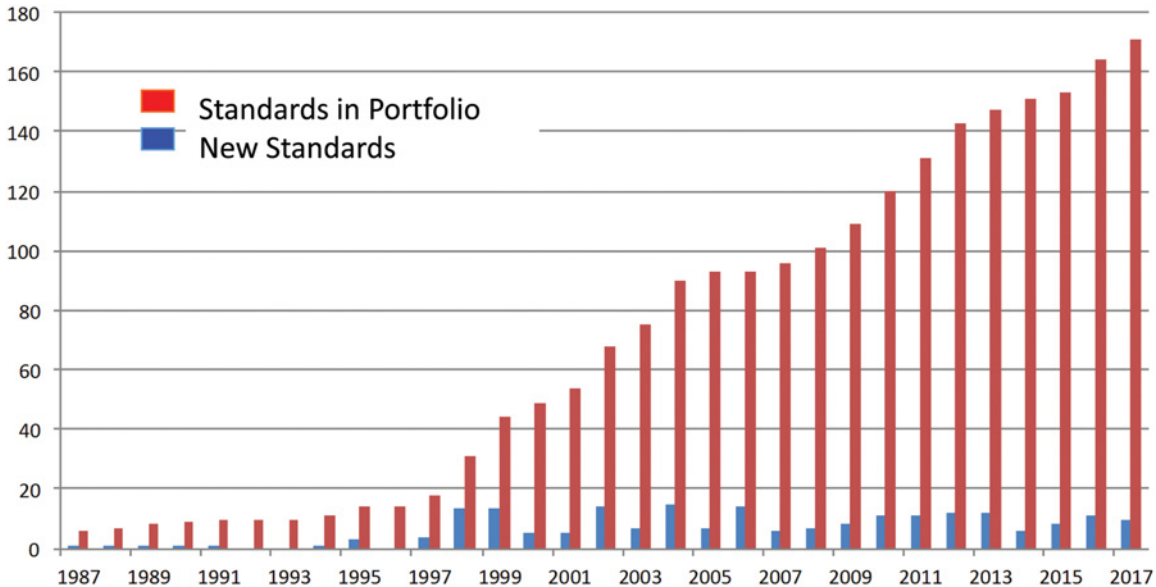

Figure 4.3 The evolution of standards SC7 [SUR 17].

The majority of software engineering standards of SC7 describe proven practices such as configuration management and quality assurance (QA) practices, while a small number of standards, such as ISO 25000 presented in the previous chapter, describe product requirements.

The ISO/IEC/IEEE 24765 [ISO 17a] glossary will be used as the reference for most of the definitions in this book. When a term is not defined in the ISO 24765, the definition of another standard will be used, such as ISO 9001, the IEEE standard or CMMI® for Development [SEI 10a].

ISO/IEC/IEEE 24765—Systems and Software Engineering Vocabulary [ISO 17a].

An online glossary can be consulted and downloaded at the following address: www.computer.org/sevocab

In recent years, some software engineering standards in the portfolio of SC7 saw their scope expanded as the practices described can be applied to a wider area than software engineering alone. For example, the scope of software engineering standards of verification and validation, risk management, and configuration management has been extended to cover the field of systems engineering that develop products which often include hardware (e.g., electronic, mechanical, and optical) and software. Thus, a greater number of engineers and developers use the same standards. This facilitates communication between different domains.

System

Combination of interacting elements organized to achieve one or more stated purposes.

Note 1 to entry: A system is sometimes considered as a product or as the services it provides.

Note 2 to entry: In practice, the interpretation of its meaning is frequently clarified by the use of an associative noun, e.g. aircraft system. Alternatively, the word "system" is substituted simply by a context-dependent synonym, e.g. aircraft, though this potentially obscures a system principles perspective.

Note 3 to entry: A complete system includes all of the associated equipment, facilities, material, computer programs, firmware, technical documentation, services and personnel required for operations and support to the degree necessary for self-sufficient use in its intended environment.

ISO 15288 [ISO 15c]

4.2 STANDARDS, COST OF QUALITY, AND BUSINESS MODELS

In a previous chapter, we presented the notions of cost of quality and business models. Regarding cost of quality, standards are an element of prevention costs: in other words, the costs incurred by an organization to prevent errors from happening during different stages in the development or maintenance process. Table 4.1 lists the different prevention cost elements. Purchasing, training, and standards implementation are also prevention costs.

We will briefly review the main business models in the software industry, namely (adapted from Iberle (2002) [IBE 02]):

- Custom systems written on contract: The organization makes profits by selling tailored software development services for clients.
- Custom software written in-house: The organization develops software to improve organizational efficiency.
- Commercial software: The company makes profits by developing and selling software to other organizations.
- Mass-market software: The company makes profits by developing and selling software to consumers.
- Commercial and mass-market firmware: The company makes profits by selling software in embedded hardware and systems.

The standards are commonly used in the following business models: custom systems written on contract, mass-market software and commercial and mass-market firmware. In these business models, standards are used to optimally manage development and minimize errors and risks. As for the "Custom systems written on contract" business model, it is the client who will decide whether or not to impose standards.

Table 4.1 Prevention Costs (Adapted from Krasner (1998) [KRA 98])

Major category	Subcategories	Definition	Typical cost
Prevention costs	Quality basis definition.	Efforts to define quality, set quality goals, standards, and thresholds.Quality trade-off analysis.	Defining release criteria for acceptance testing and related quality standards.
	Project and process-oriented interventions.	Efforts to prevent poor product quality or improve process quality.	Training, process improvements, metrics collection, and analysis.